



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第四章 数组、串与广义表



- 一维数组与多维数组
- 特殊矩阵
- 稀疏矩阵
- 字符串
- 广义表



4.4 字符串 (String)

- 字符串是 n (≥ 0) 个字符组成的有限序列,
记作 $S : "c_0c_1c_2 \dots c_{n-1}"$
其中, S 是串名字
“ $c_0c_1c_2 \dots c_{n-1}$ ”是串值
(默认包含串结束符 ‘\0’)
 c_i 是串中字符
 n 是串的长度, $n = 0$ 称为空串。
- 例如, $S = \text{“Nanjing University”}$ 。
- 注意: 空白串和空串不同, 例如 “ ” 和 “” 分别表示长度为1的空白串和长度为0的空串。



- 串中任意个连续字符组成的子序列称为该串的子串，包含子串的串相应地称为主串。
- 设A和B分别为
A = “This is a string” B = “is”
则 B 是 A 的子串，A 为主串。
B 在 A 中出现了两次，首次出现所对应的主串位置是2（从0开始），因此称 B 在 A 中的位置为2。
- 特别地，空串是任意串的子串，任意串是其自身的子串。



字符串的类定义



```
#ifndef ASTRING_H    //定义在文件“Astring.h”中
#define ASTRING_H
#define defaultSize = 128;    //字符串的最大长度
class AString {
//对象: 零个或多个字符组成的一个有限序列。
private:
    char *ch;                //串存放数组
    int curLength;            //串的实际长度
    int maxSize;              //存放数组的最大长度
```



public:

```
AString(int sz = defaultSize);    //构造函数
AString(const char *init );       //构造函数
AString(const AString& ob);        //复制构造函数
~AString() {delete [ ]ch; }        //析构函数
int Length() const { return curLength; } //求长度
AString& operator() (int pos, int len); //求子串
bool operator == (AString& ob) const
{ return strcmp (ch, ob.ch) == 0; }
//判串相等. 若串相等, 则函数返回true
bool operator != (AString& ob) const
{ return strcmp (ch, ob.ch) != 0; }
//判串不等. 若串不相等, 则函数返回true
```



```
bool operator ! () const { return curLength == 0; }
```

//判串空否. 若串空, 则函数返回true

```
AString& operator = (AString& ob);    //串赋值
```

```
AString& operator += (AString& ob);   //串连接
```

```
char& operator [ ] (int i);          //取第 i 个字符
```

```
int Find (AString& pat, int start) const; //串匹配
```

```
};
```



字符串的构造函数



```
AString::AString(int sz) {  
//构造函数: 创建一个空串  
    maxSize = sz;  
    ch = new char[maxSize+1];    //创建串数组  
    if (ch == NULL)  
        { cerr << “存储分配错!\n”; exit(1); }  
    curLength = 0;  
    ch[0] = '\0';  
};
```




字符串的构造函数



```
AString::AString(const char *init) {  
    //复制构造函数: 从已有字符数组*init复制  
    int len = strlen(init);  
    maxSize = (len > defaultSize) ? len : defaultSize;  
    ch = new char[maxSize+1];    //创建串数组  
    if (ch == NULL)  
        { cerr << “存储分配错 ! \n”; exit(1); }  
    curLength = len;              //复制串长度  
    strcpy(ch, init);             //复制串值  
};
```



字符串的复制构造函数



```
AString :: AString(const AString& ob) {
```

```
//复制构造函数: 从已有串ob复制
```

```
    maxSize = ob.maxSize;           //复制串最大长度
```

```
    ch = new char[maxSize + 1];     //创建串数组
```

```
    if (ch == NULL)
```

```
        { cerr << “存储分配错! \n”; exit(1); }
```

```
    curLength = ob.curLength;       //复制串长度
```

```
    strcpy(ch, ob.ch);              //复制串值
```

```
};
```



字符串重载操作的使用示例



序号	重载操作	操作	使用示例 (设使用操作的当前串为S: “good”)
1	<code>() (int pos, int len)</code>	取子串	<code>S1 = S(1, 3)</code> , //S1结果为“ood”
2	<code>== (const AString& ob)</code>	判两串相等	<code>S == S1</code> , //若S与S1相等, 结果为true, 否则为false
3	<code>!= (const AString& ob)</code>	判两串不等	<code>S != S1</code> , //若S与S1不等, 结果为true, 否则为false
4	<code>!()</code>	判串空否	<code>!S</code> , //若串S为空, 结果为true, 否则为false



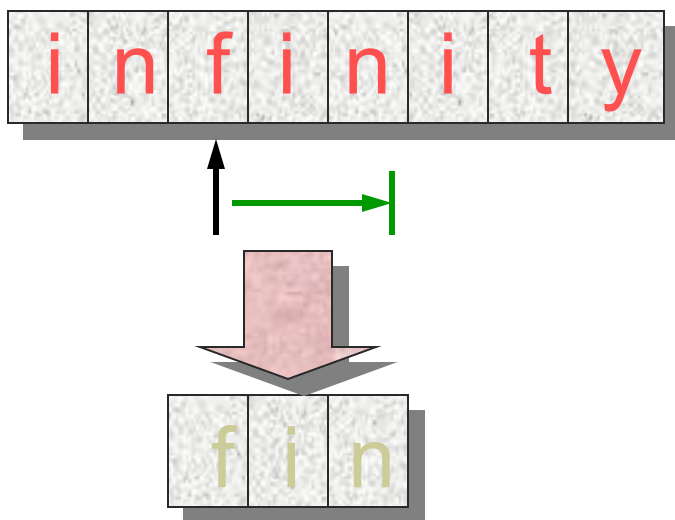
序号	重载操作	操作	使用示例 (设使用操作的当前串为S: “good”)
5	= (const AString& ob)	串赋值	S1 = S, // S1 结果为“good”
6	+= (const AString& ob)	串连接	若设S1为“morning”, 执行 S += S1, // S 结果为“good morning”
7	[] (int i)	取第 i 个字符	S [3], //取出字符为“d”



提取子串的算法示例

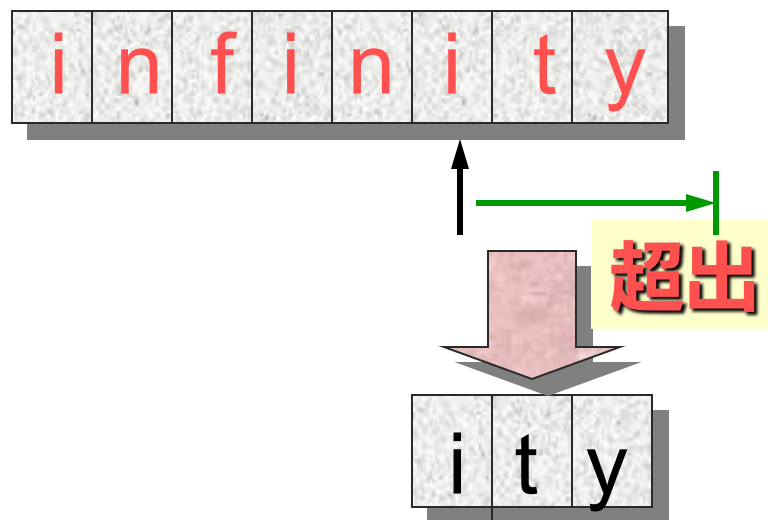


pos = 2, len = 3



$\text{pos} + \text{len} - 1$
 $\leq \text{curLength} - 1$
可以全部提取

pos = 5, len = 4



$\text{pos} + \text{len} - 1$
 $\geq \text{curLength}$
只能从pos取到串尾



串重载操作：提取子串



```
AString AString::operator () (int pos, int len) {  
    //从串中第 pos 个位置起连续提取 len 个字符形成  
    //子串返回  
    AString temp;                                //建立空串对象  
    if (pos >= 0 && pos+len-1 < maxSize && len > 0)  
    {                                              //提取子串  
        if (pos+len-1 >= curLength)  
            len = curLength - pos;              //调整提取字符数  
        temp.curLength = len;                    //子串长度  
    }  
}
```



```
for (int i = 0, j = pos; i < len; i++, j++)  
    temp.ch[i] = ch[j];           //传送串数组  
temp.ch[len] = '\0';             //子串结束  
}  
return temp;  
};
```

- 例：串 `st = “university”`, `pos = 3`, `len = 4`
使用示例 `subSt = st(3, 4)`
提取子串 `subSt = “vers”`



串重载操作：串赋值



```
AString& AString::operator = (const AString& ob) {  
    if (&ob != this) {        //若两个串相等为自我赋值  
        delete []ch;  
        ch = new char[maxSize+1];    //重新分配  
        if (ch == NULL)  
            { cerr << “存储分配失败!\n”; exit(1); }  
        curLength = ob.curLength; strcpy(ch,ob.ch);  
    }  
    else cout << “字符串自身赋值出错!\n”;  
    return this;  
};
```




串重载操作：取串的第i个字符



```
char AString::operator [ ] (int i) {  
    //串重载操作：取当前串*this的第i个字符  
    if (i < 0 || i >= curLength)  
        { cout << “字符串下标超界!\n”; exit(1); }  
    return ch[i];  
};
```

- 例：串 st = “university”，
使用示例 `newSt = st;` //`newSt` = “university”
 `newChar = st[1];` //`newChar` = ‘n’



串重载操作：串连接



```
AString& AString::operator += (const AString& ob)
{
    char *temp = ch;           //暂存原串数组
    int n = curLength + ob.curLength;    //串长度累加
    int m = (maxSize >= n) ? maxSize : n; //新空间大小
    ch = new char[m];
    if (ch == NULL)
        { cerr << “存储分配错!\n”; exit(1); }
    maxSize = m; curLength = n;
    strcpy(ch, temp);           //拷贝原串数组
    strcat(ch, ob.ch);          //连接ob串数组
}
```



```
delete []temp;  
return this;  
};
```

- **例：串** `st1 = “Nanjing ”`,
 `st2 = “university”`,
使用示例 `st1 += st2`;

连接结果 `st1 = “Nanjing university”`
 `st2 = “university”`



串的模式匹配



- **定义** 在主串中寻找子串（第一个字符）在串中的位置
- **词汇** 在模式匹配中，子串称为**模式 (Pattern)**，主串称为**目标 (Target)**。
- **示例** 目标 T : “Nanjing”
模式 P : “jin”
匹配结果 = 3



朴素的模式匹配



第1趟

<i>T</i>	a	b	b	a	b	a
<i>P</i>	a	b	a			

↓ $i=2$
↑ $j=2$

第3趟

<i>T</i>	a	b	b	a	b	a
<i>P</i>			a	b	a	

↓ $i=2$
↑ $j=0$

第2趟

<i>T</i>	a	b	b	a	b	a
<i>P</i>		a	b	a		

↓ $i=1$
↑ $j=0$

第4趟

<i>T</i>	a	b	b	a	b	a
<i>P</i>				a	b	a

↓ $i=6$
↑ $j=3$



朴素的模式匹配算法



```
int AString::Find(AString& pat, int start) const {  
    //在当前串中从第 start 个字符开始寻找模式 pat 在当  
    //前串中匹配的位置。若匹配成功,则函数返回首  
    //次匹配的位置, 否则返回-1。
```

```
    int i, j, n = curLength, m = pat.curLength;  
    if (m == 0) return -1;  //pat为空  
    for (i = start; i <= n-m; i++) {  
        for (j = 0; j < m; j++){  
            if (ch[i+j] != pat.ch[j]) break;  //本次失配  
        }  
        if (j == m) return i;  //pat扫描完, 匹配成功  
    }  
    return -1;  //在*this中找不到子串  
};
```



- 在**最坏情况**下，若设 n 为目标串长度， m 为模式串长度，则匹配算法最多比较 $n-m+1$ 趟，每趟比较都在比较到模式串尾部才出现不等，要做 m 次比较，总比较次数将达到 $(n-m+1)*m$ 。在多数场合下 m 远小于 n ，因此，算法的运行时间为 $O(n*m)$ 。
- 低效的原因在于每趟重新比较时，目标串的检测指针要回退。



改进的模式匹配



- 只要消除了每趟失配后为实施下一趟比较时目标指针的回退，就可以提高模式匹配效率。
- 这种处理思想是由D.E.Knuth、J.H.Morris和V.R.Pratt同时提出来的，故称为**KMP算法**。

最年轻的图灵奖得主D.E.Knuth的《计算机程序设计的艺术》系列，开始于他念博士期间，计划出七卷，第一卷《基本算法》于1968年出版，第二卷《半数字化算法》于1969年出版，第三卷《排序与搜索》于1973年出版，第四卷《组合算法》于2008年出版。



改进的模式匹配



目标 T $t_0 \quad t_1 \quad t_2 \quad \dots \quad t_{m-1} \quad \dots \quad t_{n-1}$
 $\updownarrow \quad \updownarrow \quad \updownarrow \quad \quad \quad \updownarrow$
模式 P $p_0 \quad p_1 \quad p_2 \quad \dots \quad p_{m-1}$

目标 T $t_0 \quad t_1 \quad t_2 \quad \dots \quad t_{m-1} \quad t_m \quad \dots \quad t_{n-1}$
 $\updownarrow \quad \updownarrow \quad \quad \quad \updownarrow \quad \updownarrow$
模式 P $p_0 \quad p_1 \quad \dots \quad p_{m-2} \quad p_{m-1}$

目标 T $t_0 \quad t_1 \quad \dots \quad t_i \quad t_{i+1} \dots \quad t_{i+m-2} \quad t_{i+m-1} \dots \quad t_{n-1}$
 $\parallel \quad \parallel \quad \quad \quad \parallel \quad \parallel$
模式 P $p_0 \quad p_1 \quad \dots \quad p_{m-2} \quad p_{m-1}$



$$\begin{array}{cccccccccccc}
 \text{T} & t_0 & t_1 & \dots & t_{s-1} & t_s & t_{s+1} & t_{s+2} & \dots & t_{s+j-1} & t_{s+j} & t_{s+j+1} & \dots & t_{n-1} \\
 & & & & & \parallel & \parallel & \parallel & \parallel & \parallel & \times & & & \\
 \text{P} & & & & & p_0 & p_1 & p_2 & \dots & p_{j-1} & p_j & & &
 \end{array}$$

则有 $t_s t_{s+1} t_{s+2} \dots t_{s+j-1} = p_0 p_1 p_2 \dots p_{j-1} \quad (1)$

下一趟，为使模式 P 与目标 T 匹配，必须满足

$$p_0 p_1 p_2 \dots p_{j-1} \dots p_{m-1} = t_{s+1} t_{s+2} t_{s+3} \dots t_{s+j} \dots t_{s+m}$$

如果 $p_0 p_1 \dots p_{j-2} \neq p_1 p_2 \dots p_{j-1} \quad (2)$

则立刻可以断定

$$p_0 p_1 \dots p_{j-2} \neq t_{s+1} t_{s+2} \dots t_{s+j-1}$$

下一趟必不匹配

$$p_0 \quad p_1 \quad \dots \quad p_{j-2}$$



同样，若 $p_0 p_1 \dots p_{j-3} \neq p_2 p_3 \dots p_{j-1}$
则再下一趟也不匹配，因为有

$$p_0 p_1 \dots p_{j-3} \neq t_{s+2} t_{s+3} \dots t_{s+j-1}$$

直到对于某一个“k”值，使得

$$p_0 p_1 \dots p_{k+1} \neq p_{j-k-2} p_{j-k-1} \dots p_{j-1}$$

且

$$p_0 p_1 \dots p_k = p_{j-k-1} p_{j-k} \dots p_{j-1}$$

则

$$p_0 p_1 \dots p_k = t_{s+j-k-1} t_{s+j-k} \dots t_{s+j-1}$$

|| || ||

$$p_{j-k-1} p_{j-k} \dots p_{j-1}$$

下一趟可以直接用 p_{k+1} 与 t_{s+j} 继续比较。



k 的确定方法



- Knuth 等人发现，对于不同的 j （失配位置）， k 的取值不同，它仅依赖于模式 P 本身前 j 个字符的构成，与目标无关。
- 可以用一个 **next 特征向量** 来确定：当模式 P 中第 j 个字符与目标 T 中相应字符失配时，模式 P 中应当由哪个字符（设为第 $k+1$ 个）与目标中刚失配的字符重新继续进行比较。



- 设模式 $P = p_0p_1\cdots p_{m-2}p_{m-1}$, next特征向量定义如下:

$$\text{next}(j) = \begin{cases} -1, & j = 0 \\ k + 1, & 0 \leq k < j-1 \text{ 且使得 } p_0p_1\cdots p_k = p_{j-k-1}p_{j-k}\cdots p_{j-1} \text{ 的最大整数} \\ 0, & \text{其他情况} \end{cases}$$

- 示例

j	0	1	2	3	4	5	6	7
P	a	b	a	a	b	c	a	c
next(j)	-1	0	0	1	1	2	0	1



利用next特征向量进行匹配处理



- 若设在进行某一趟匹配比较时在模式 P 的第 j 位失配:
 - ◆ 如果 $j > 0$, 那么在下一趟比较时模式串 P 的起始比较位置是 $p_{\text{next}(j)}$, 目标串 T 的指针不回溯, 仍指向上一趟失配的字符;
 - ◆ 如果 $j = 0$, 则目标串指针 T 进一, 模式串指针 P 回到 p_0 , 继续进行下一趟匹配比较。



运用KMP算法的匹配过程

第1趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$
 $\times j=1 \Rightarrow \text{next}(1) = 0$, 下次 p_0

第2趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$
 $\times j=0 \Rightarrow$ 下次 p_0 , 目标指针进 1

第3趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$
 $\times j=5 \Rightarrow \text{next}(5) = 2$,
下次 p_2

第4趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $(a b) a a b c a c \checkmark$



用KMP算法实现快速匹配算法



```
int AString::fastFind(AString& pat, int start, int next[]) const
```

```
{  
    //从 start 开始寻找 pat 在 this 串中匹配的位置。若找到，函数返回 pat 在  
    //this 串中开始位置，否则函数返回-1。数组next[ ] 存放 pat 的next[j] 值。
```

```
    int posP = 0, posT = start; //两个串的扫描指针
```

```
        int lengthP = pat.curLength; //模式串长度
```

```
        int lengthT = curLength; //目标串长度
```

```
        while (posP < lengthP && posT < lengthT)
```

```
        { if (posP == -1 || pat.ch[posP] == ch[posT])
```

```
            { posP++; posT++; } //对应字符匹配
```

```
            else posP = next[posP]; //求pat下趟比较位置
```

```
        }
```

```
        if (posP < lengthP) return -1; //匹配失败
```

```
        else return posT-lengthP; //匹配成功
```

```
};
```




- 此算法的时间复杂度取决于 while 循环。由于是无回溯的算法，执行循环时，目标串字符比较有进无退，要么执行 posT 和 posP 进 1，要么查找 next[] 数组进行模式位置的右移，然后继续向后比较。
- 若设 n 为目标串长度， m 为模式串长度，字符的比较次数最多为 $O(n)$ ，不超过目标串的长度。



next特征向量的计算



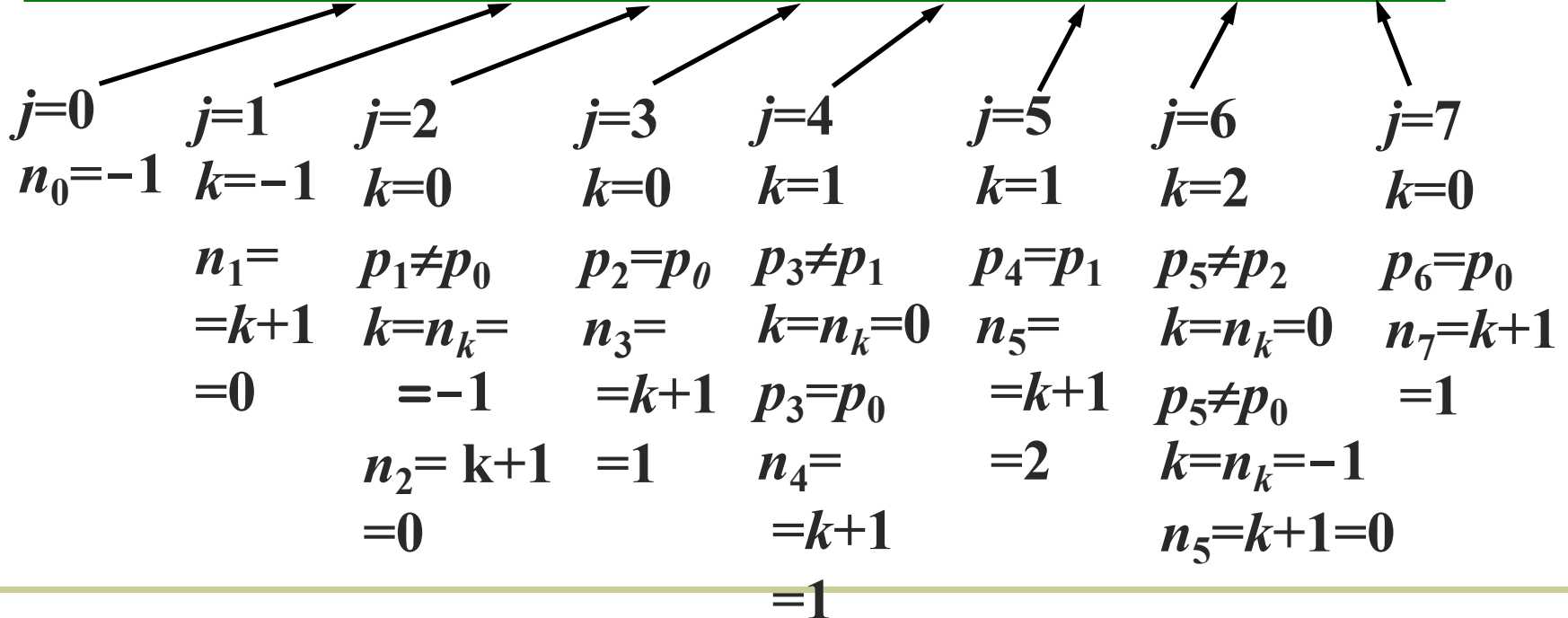
- 设模式 $P = p_0 p_1 p_2 \dots p_{m-1}$ 由 m 个字符组成, 而next特征向量为 $\text{next} = n_0 n_1 n_2 \dots n_{m-1}$, 表示了模式的字符分布特征。
- next特征向量从0, 1, 2, ..., $m-1$ 逐项递推计算:
 - ① 当 $j = 0$ 时, $n_0 = -1$ 。设 $j > 0$ 时 $k = n_{j-1}$:
 - ② 当 $k == -1$ 或 $(j > 0 \text{ 且 } p_{j-1} == p_k)$, 则 $n_j = k+1$ 。
 - ③ 当 $p_{j-1} \neq p_k$ 且 $k \neq -1$, 令 $k = n_k$, 并让③循环直到条件不满足, $n_j = k+1$



• 以前面的例子说明:



j	0	1	2	3	4	5	6	7
P	a	b	a	a	b	c	a	c
$n[j]$	-1	0	0	1	1	2	0	1





对当前串计算next特征向量的算法



```
void AString::getNext(int next[]) {  
    int j = 0, k = -1, lengthP = curLength;  
    next[0] = -1; j = 1;  
    while (j < lengthP) {                //计算next[j]  
        k = next[j-1];  
        while(1) {  
            if ( k == -1 || ch[j-1] == ch[k]) {  
                k++;  
                next[j] = k;  
                j++;  
                break;  
            }  
            else k = next[k];  
        }  
    }  
};
```



要理解 next 数组，先要理解字符串的 **真前缀** 和 **真后缀**。

- **前缀**：指从字符串开头开始的任意连续子串（不包括字符串本身）。
- **后缀**：指以字符串结尾的任意连续子串（不包括字符串本身）。

例如，对于字符串 "ABABA"：

- 真前缀： "A", "AB", "ABA", "ABAB"
- 真后缀： "A", "BA", "ABA", "BABA"
- 最长公共前后缀： "ABA"（长度为3）



`next` 数组是针对于**模式串 (P)** 预先计算出来的一个数组。

- `next[i]` 的定义是：对于模式串 `P` 的子串 `P[0..i]`，其最长公共前后缀的长度。

以模式串 `P = "ABABAC"` 为例，我们手动计算其 `next` 数组：

i	子串 P[0..i]	真前缀	真后缀	最长公共前后缀	next[i]
0	"A"	(无)	(无)	(无)	0
1	"AB"	"A"	"B"	(无)	0
2	"ABA"	"A", "AB"	"A", "BA"	"A"	1
3	"ABAB"	"A", "AB", "ABA"	"B", "AB", "BAB"	"AB"	2
4	"ABABA"	"A", ..., "ABAB"	"A", ..., "BABA"	"ABA"	3
5	"ABABAC"	"A", ..., "ABABA"	"C", ..., "BABAC"	(无)	0

所以，对于 `P = "ABABAC"`，其 `next` 数组为： `[0, 0, 1, 2, 3, 0]`。





构建 `next` 数组本身也是一个字符串匹配过程，可以看作模式串与自身进行匹配。

算法步骤（`P` 的下标从 0 开始）：

1. `next[0] = 0`。（第一个字符没有真前后缀）
2. 设 `i = 1`（指向当前要计算的位置），`len = 0`（指向上一个位置的最长公共前后缀长度，即 `next[i-1]`）。
3. 循环 `i` 从 1 到 `m-1`（`m` 为模式串长度）：
 - **while (`len > 0 && P[i] != P[len]`)**：
 - 如果当前字符不匹配，`len` 需要回退到 `next[len-1]`。这类似于匹配过程中的回退。
 - **if (`P[i] == P[len]`)**：
 - 如果匹配，最长公共前后缀长度增加 1，即 `len++`。
 - `next[i] = len`。

这个过程的时间复杂度是 $O(m)$ 。



题目：

已知主串 $S = \text{"ABCABCDABABCDABCDABDE"}$ ，模式串 $P = \text{"ABCDABD"}$ 。

1. 请计算模式串 P 的 next 数组。
2. 使用 KMP 算法模拟在 S 中查找 P 的过程，指出匹配成功时在主串中的起始位置。

题目：

对于模式串 $P = \text{"ABACABAD"}$ 。

1. 求出其 next 数组。
2. 求出其优化后的 nextval 数组，并解释为什么需要优化。



2. 求 nextval 数组

nextval 是对 next 的优化。规则如下：

1. $\text{nextval}[0] = -1$ 或 0 （约定俗成，通常为-1表示特殊情况，这里沿用0也可，但优化逻辑不变。通常教材用-1起始，但原理核心是避免与失配字符相同）。我们按常见定义 $\text{nextval}[0] = -1$ 。
2. 对于 $i > 0$:
 - 如果 $P[i] == P[\text{next}[i]]$ ，则 $\text{nextval}[i] = \text{nextval}[\text{next}[i]]$ 。
 - 否则， $\text{nextval}[i] = \text{next}[i]$ 。

计算过程：

- $i=0$: $\text{nextval}[0] = -1$ (约定)
- $i=1$: $\text{next}[1]=0$, 比较 $P[1]('B')$ 和 $P[0]('A')$ 。不同 $\rightarrow \text{nextval}[1] = \text{next}[1] = 0$
- $i=2$: $\text{next}[2]=1$, 比较 $P[2]('A')$ 和 $P[1]('B')$ 。不同 $\rightarrow \text{nextval}[2] = \text{next}[2] = 1$
- $i=3$: $\text{next}[3]=0$, 比较 $P[3]('C')$ 和 $P[0]('A')$ 。不同 $\rightarrow \text{nextval}[3] = \text{next}[3] = 0$
- $i=4$: $\text{next}[4]=1$, 比较 $P[4]('A')$ 和 $P[1]('B')$ 。不同 $\rightarrow \text{nextval}[4] = \text{next}[4] = 1$
- $i=5$: $\text{next}[5]=2$, 比较 $P[5]('B')$ 和 $P[2]('A')$ 。不同 $\rightarrow \text{nextval}[5] = \text{next}[5] = 2$
- $i=6$: $\text{next}[6]=3$, 比较 $P[6]('A')$ 和 $P[3]('C')$ 。不同 $\rightarrow \text{nextval}[6] = \text{next}[6] = 3$
- $i=7$: $\text{next}[7]=0$, 比较 $P[7]('D')$ 和 $P[0]('A')$ 。不同 $\rightarrow \text{nextval}[7] = \text{next}[7] = 0$

所以 $\text{nextval} = [-1, 0, 1, 0, 1, 2, 3, 0]$ 。在这个特例中，nextval 与 next 相同，因为每次回退后的字符都与原失配字符不同。

1. 求 next 数组

$P = \text{"ABACABAD"}$

- $i=0$, "A" $\rightarrow \text{next}[0]=0$
 - $i=1$, "AB" $\rightarrow$ 无公共 $\rightarrow \text{next}[1]=0$
 - $i=2$, "ABA" $\rightarrow$ 公共"A" $\rightarrow \text{next}[2]=1$
 - $i=3$, "ABAC" $\rightarrow$ 无公共 $\rightarrow \text{next}[3]=0$
 - $i=4$, "ABACA" $\rightarrow$ 公共"A" $\rightarrow \text{next}[4]=1$
 - $i=5$, "ABACAB" $\rightarrow$ 公共"AB" $\rightarrow \text{next}[5]=2$
 - $i=6$, "ABACABA" $\rightarrow$ 公共"ABA" $\rightarrow \text{next}[6]=3$
 - $i=7$, "ABACABAD" $\rightarrow$ 无公共 $\rightarrow \text{next}[7]=0$
- $\text{next} = [0, 0, 1, 0, 1, 2, 3, 0]$



为什么需要优化?

考虑一个极端例子 $P = \text{"AAAAA"}$ 。

- $\text{next} = [0, 1, 2, 3, 4]$ 。
- 如果在 $j=4$ 处失配, 根据 next 数组, j 会回退到 3, 2, 1, 0。但 $P[4] = P[3] = P[2] = P[1] = P[0] = \text{'A'}$ 。既然都是在字符 'A' 上失配, 那么回退多次并重复比较相同的字符是低效的。
- 优化后的 nextval 数组: $[-1, -1, -1, -1, -1]$ 。一次回退就直接跳到开头, 避免了多次无意义的比较。这就是 nextval 的优化目的: **避免在模式串中相同字符处多次失配。**